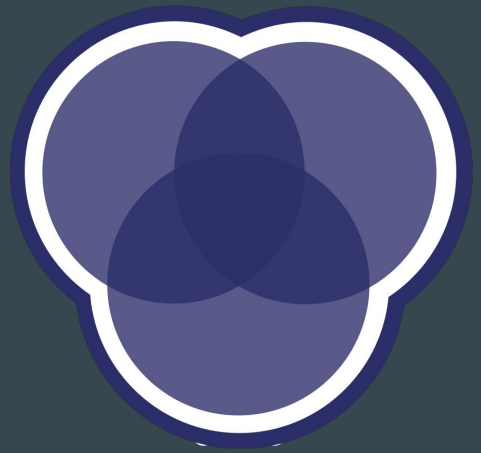


Slurm-Aware: Learned Job Scheduler

Context

The CDS cluster runs jobs through Slurm.

CURRENT SCHEDULING

FIFO and backfill use requested resources and queue state to decide which jobs run.

THE BLIND SPOT

At submission, Slurm has no reliable estimate of actual runtime or resource use. Short jobs may wait, and GPUs may sit idle.

Can predictions help us choose a better job order?

Project Goal & Scope

Build a scheduler that uses predicted job behavior to choose job priority, then compare it with Slurm.

01 JOB TRACES

Collect logs or generate realistic traces of submitted jobs.

02 PREDICT

Estimate runtime and resource needs from submission metadata.

03 SCHEDULE

Use predictions to prioritize or reorder queued jobs.

04 BENCHMARK

Compare queue wait time and GPU utilization with Slurm.

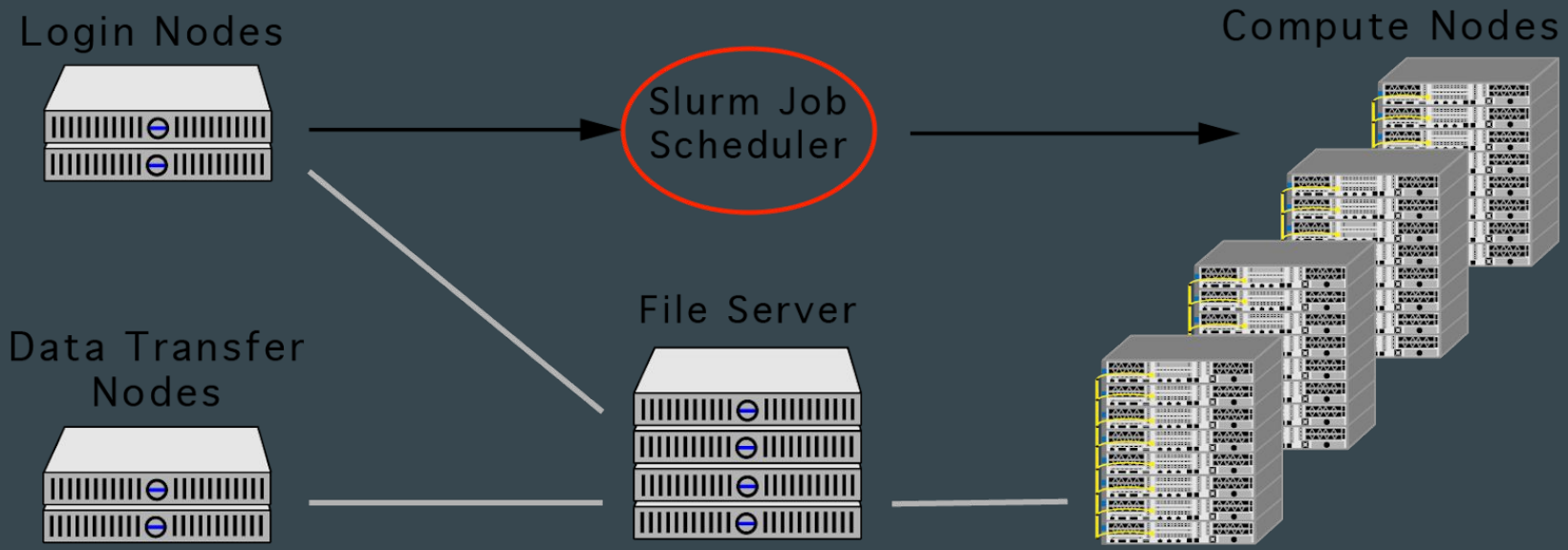
SEMESTER TARGET

Working prototype with a measurable gain in at least one benchmark metric.

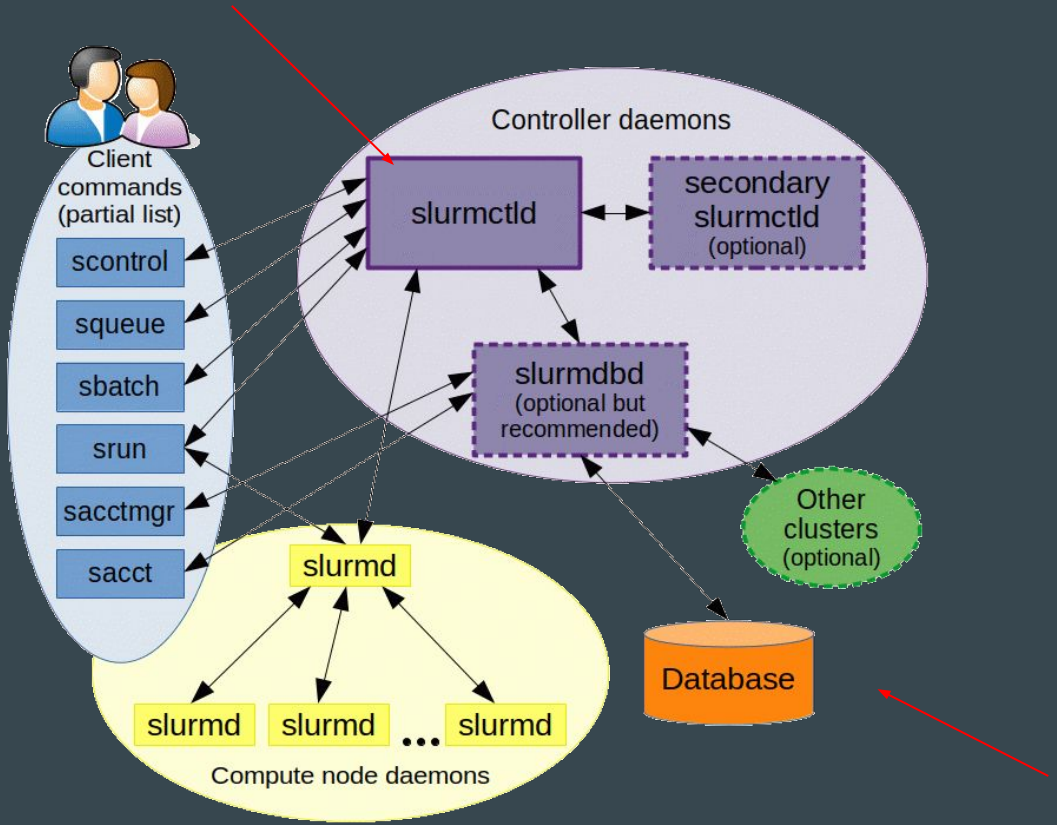
IF TIME ALLOWS

Multi-resource prediction or preemption-aware scheduling.

Overview - Slurm



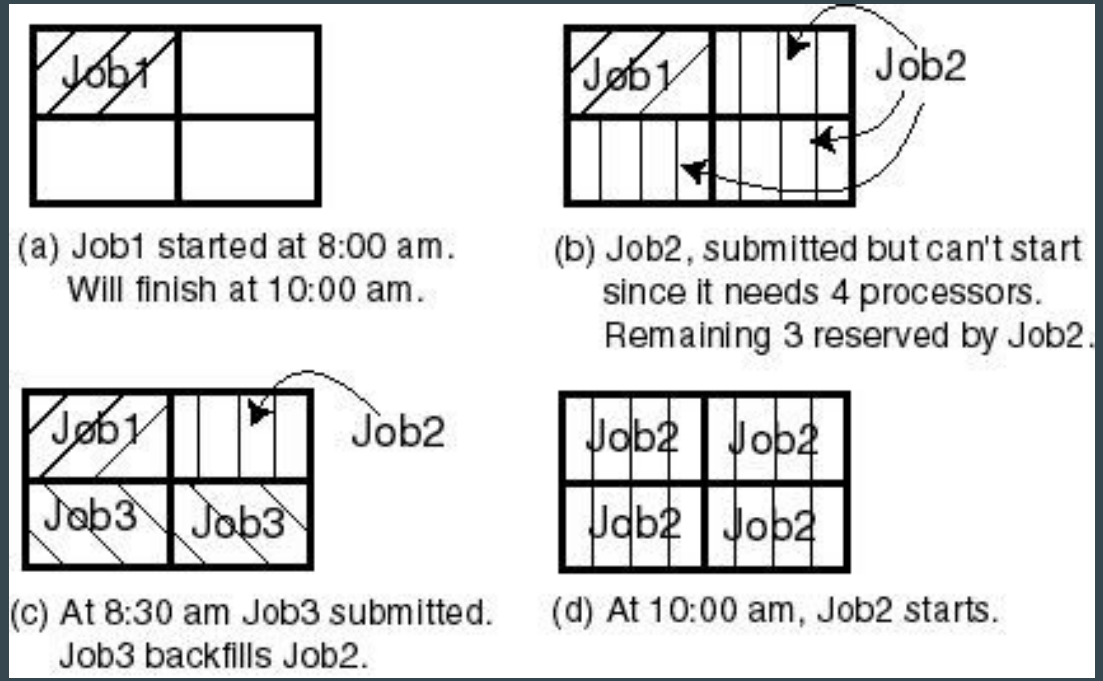
Overview - Slurm



Overview - How Slurm Schedules Jobs

- Jobs are assigned a priority and added to a partition (priority queue) to wait to be scheduled onto resources
  - **Priority → some function of how long job has been waiting, amount of time requested, etc**
- Resources are “reserved” for the highest priority jobs - guaranteed access as soon as currently running jobs are done
- Backfill
  - Smaller jobs can run on idle sections of the reserved resources

Overview - How Slurm Schedules Jobs



Overview - How Slurm Schedules Jobs

- Preemption can be used to immediately free resources held by low-priority jobs for a higher priority one
  - Gang Scheduling (like OS)
  - Suspension (resume interrupted task after)
  - Requeue
  - Cancel
- Can enable jobs in specific certain partitions and to be interrupted while running

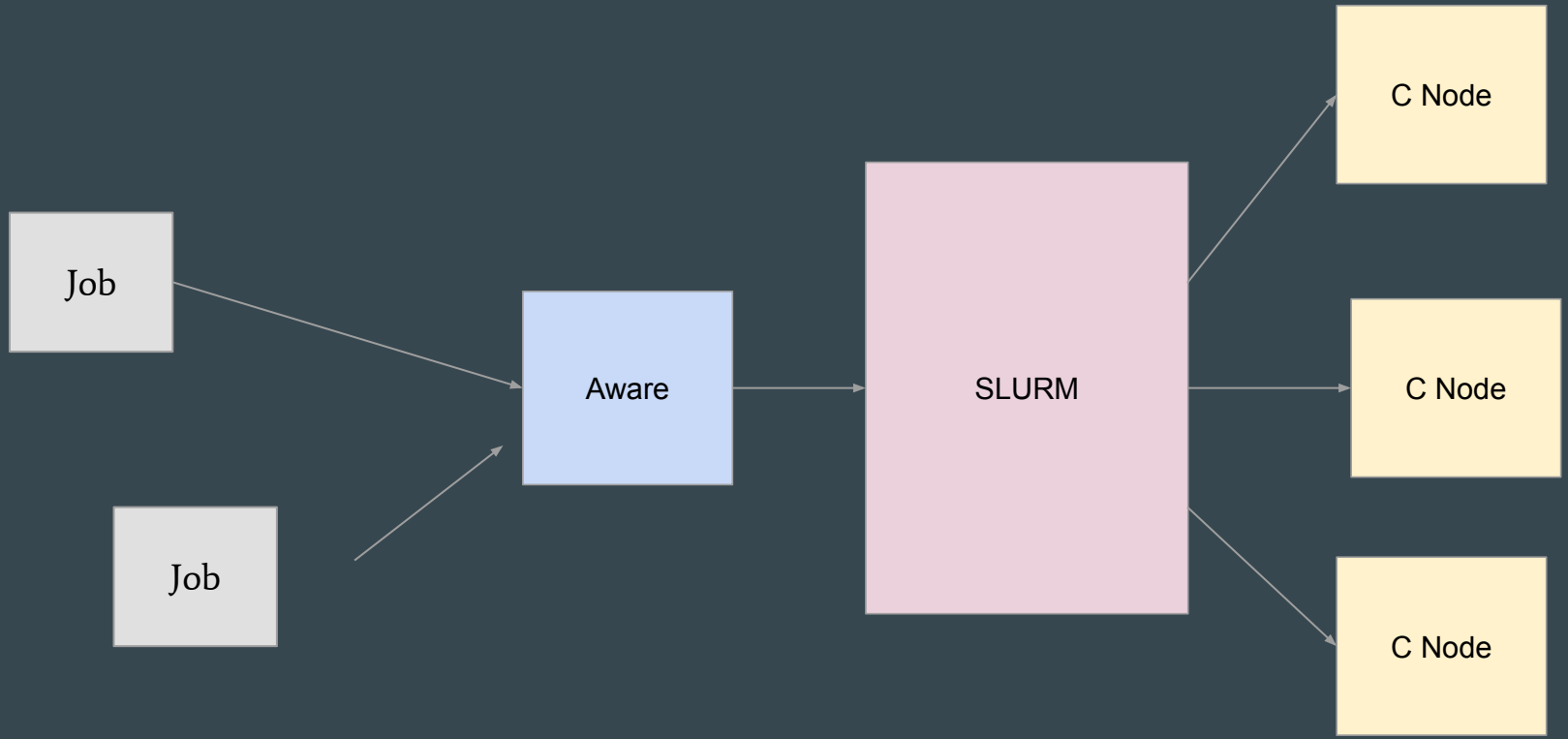
Overview - How Slurm Assigns Priority

Job Priority Factors In General §

The job's priority at any given time will be a weighted sum of all the factors that have been enabled in the slurm.conf file. Job priority can be expressed as:

```
Job_priority =
        site_factor +
        (PriorityWeightAge) * (age_factor) +
        (PriorityWeightAssoc) * (assoc_factor) +
        (PriorityWeightFairshare) * (fair-share_factor) +
        (PriorityWeightJobSize) * (job_size_factor) +
        (PriorityWeightPartition) * (priority_job_factor) +
        (PriorityWeightQOS) * (QOS_factor) +
        SUM(TRES_weight_cpu * TRES_factor_cpu,
            TRES_weight_<type> * TRES_factor_<type>,
            ...)
        - nice_factor
```

The Aware Part



The Aware Part

- Assign priorities by potentially defining an extended job description
- Use learned weights to compute job priorities, rather than hand picked ones
- Experiment a lot

- As for execution, we will figure this out as we go! Some ideas below...
  - Write Slurm plugins
  - Build external service that sits in front of Slurm and reorders jobs itself
  - Something else

What do you guys want?

Meeting times

More systems stuff vs more ml stuff

Personal goals out the project

GET THE CHATGPT PLAN + SEND GITHUBS

https://chatgpt.com/students/2026/



ALSO SEND YOUR GITHUBS INTO SLACK